

Primitive-Ring-Bound Refinement Specification

Stage 10C: Full Downstream Rebuilds, Stable Scientific Keys, and Tested-Suffix Stabilization

mdstats

2026-07-19

Contents

Purpose and stage boundary	2
Motivation	2
External method and original construction	2
Scientific and persistence definitions	3
Primitive-bound rebuild	3
Stable refinement record	3
Stable record categories	4
Primitive ring	4
Ring orbit	4
Ring strength	4
Scientific face	4
Compatibility system	4
Master scientific complex	4
Master partition	5
Stage-10B search	5
Natural tiling	5
Essential ring	5
Snapshot completeness	5
Transition comparison	5
Primitive-ring monotonicity	6
Transition status	6
Tested-suffix stabilization	6
Public API	6
Resources	6
Build one snapshot	6
Compare two bounds	7
Build a report from snapshots	7
Execute full rebuilds	7
Algorithm	7
Serialization and tamper resistance	8
Edge cases and failure semantics	8
Same scientific result, new catalog digest	8
New longer primitive rings with no tiling effect	8
Strength changes after the component domain grows	8
Dense ring IDs are reordered	8
Primitive ring disappears under complete refinement	8
Incomplete upper bound appears unchanged	8
Different primitive cell, net view, or embedding	8
Multiple master refinements	8
No Stage-10B search	9
Focused validation requirements	9

Deferred work	9
References	9

Purpose and stage boundary

Stage 10C makes the primitive-ring upper bound an explicit **full-rebuild boundary**. For each requested bound K , the caller constructs a fresh, source-consistent downstream stack:

```
primitive rings
-> source-bound ring index
-> induced ring symmetry
-> bounded ring strength
-> embedded face candidates
-> witness compatibility systems
-> master periodic cell complexes
-> exact partition certificates
-> Stage-10B searches
-> aggregate Stage-10A natural-tiling catalog
```

Stage 10C validates that every downstream object belongs to the current primitive-ring catalog, reduces the results to stable scientific records that do not depend on dense local IDs or catalog-bound digests, and compares consecutive bounds.

Runtime/API target:

mdstats 0.19.26a0

Primary module:

mdstats/analysis/natural_tiling_refinement.py

The first backend does **not** incrementally patch old results. It also does not construct master refinements automatically, infer mappings between different net views or primitive cells, prove stabilization for all untested bounds, or run the LTA end-to-end gate. Automatic master-refinement generation remains deferred; LTA validation is Stage 10D.

Motivation

A primitive-ring catalog is finite only relative to its requested upper bound. Increasing the bound can introduce new primitive rings that alter:

- symmetry orbits of the represented ring family;
- bounded strong/weak decompositions;
- admissible face candidates and their compatibility constraints;
- master refinements and maximal face selections;
- tile complexes, properness, and natural-tiling multiplicity; and
- the set of essential rings.

Dense `ring_id`, face indices, tile indices, and source-bound SHA-256 digests are local to one rebuild. Comparing them directly would report false changes whenever an otherwise identical result is reconstructed against a new catalog digest. Conversely, reusing an old downstream object after increasing K could suppress real changes.

Stage 10C therefore applies two strict rules:

1. **rebuild, never patch**: every source-bound result must name the current primitive-ring catalog; and
2. **compare stable science, never local storage identity**: revisions are compared through canonical ring keys, translated face identities, and normalized scientific cell-complex records.

External method and original construction

The natural-tiling scientific context remains that of Blatov, Delgado-Friedrichs, O’Keeffe, and Proserpio [1]. Translation-labelled periodic ring and cell identities remain compatible with the vector method of Chung, Hahn, and Klee [2].

No new published refinement algorithm is imported in Stage 10C. The following are project-specific `mdstats` constructions:

- treating K as a transactional invalidation boundary;

- source-binding validation for every downstream result;
- category-specific normalization into stable scientific records;
- exact added/removed/modified comparison by stable keys;
- primitive-ring monotonicity validation under complete increasing bounds;
- multidimensional unresolved/invalid transition semantics; and
- tested-suffix stabilization that explicitly avoids extrapolation beyond the supplied sequence.

Scientific and persistence definitions

Primitive-bound rebuild

For one requested bound K , a `PrimitiveBoundBuild` contains:

```
PrimitiveBoundBuild(
    primitive_ring_bound=K,
    periodic_net_view_digest=...,
    periodic_net_embedding_digest=...,
    ring_index=...,
    ring_symmetry=...,
    strength_catalog=...,
    face_certificates=...,
    compatibility_systems=...,
    master_complexes=...,
    master_partitions=...,
    search_results=...,
    catalog=...,
    unresolved_reasons=...,
)
```

It is transient because it holds full source objects. Construction validates:

- `ring_index.catalog.options.max_ring_size == K`;
- primitive completeness does not claim a bound above K ;
- ring symmetry names the current ring-catalog digest and exact ring-key order;
- strength results name the current ring-catalog digest;
- every face certificate names the current view, embedding, and ring catalog;
- every master complex names the current view, embedding, and ring catalog;
- every partition certifies one supplied current-bound master complex;
- every Stage-10B search names supplied current-bound strength, compatibility, complex, and partition records; and
- the aggregate natural-tiling catalog equals the canonical union of all rebuilt Stage-10B catalogs.

A mismatch is an input error, not an unresolved scientific result. In particular, an old face certificate or search result cannot be reused merely because its dense indices still happen to be valid.

Stable refinement record

Each scientific item is reduced to

```
StableRefinementRecord(
    category=...,
    identity_json=...,
    state_json=...,
    key_digest=...,
    state_digest=...,
)
```

The identity and state strings are canonical JSON. The stable key is

$$h = \text{SHA256}(\text{canonical_json}(\text{category}, \text{identity})).$$

`key_digest` identifies the scientific object across bounds. `state_digest` identifies its classification or evidence state at one bound.

The implementation rejects noncanonical JSON and inconsistent stored digests.

Stable record categories

Primitive ring

Identity:

PrimitiveRingKey

State:

ring size

The key is the canonical decorated edge-token cycle. Dense ring_id is excluded.

Ring orbit

Identity:

sorted set of PrimitiveRingKey values in the orbit

State:

sorted stabilizer orders of the represented members

Orbit positions and operation-table row indices are excluded.

Ring strength

Identity:

canonical target PrimitiveRingKey

State retains the bounded domain, status, diagnostics, and witness decomposition, after removing source catalog digests, topology digests, and record digests. Thus a genuine STRONG_IN_DOMAIN → WEAK_CERTIFIED change is reported, while a new source-catalog hash alone is ignored.

Scientific face

For a face placement f , the stable identity is

$$F(f) = (d_E, k_R, \mathbf{t}, \sigma),$$

where d_E is the periodic embedding digest, k_R is the primitive-ring key, $\mathbf{t} \in \mathbb{Z}^3$ is the placement image shift, and $\sigma \in \{-1, +1\}$ is its orientation.

State retains finite triangulation exhaustion, witness geometry, framework contacts, rejections, and placement status after source-bound fields are removed.

Compatibility system

Identity:

sorted stable face-domain keys

Assignments and constraints are normalized by replacing source-bound face, certificate, and witness digests with stable face/witness keys. Unordered domains are sorted after replacement. Geometric candidate-set evidence remains in state.

Master scientific complex

A stable complex identity contains:

- fixed view and embedding identity;
- quotient cell counts;
- the translation-labelled ∂_1 operator;
- each ∂_2 face boundary keyed by stable face identity;
- each tile shell expressed through stable face keys, image shifts, and coefficients; and
- tile labels.

Face and tile dense indices are used only to read the source object and are then eliminated. Tile shells are canonically sorted.

Master partition

The exact auxiliary tetrahedral partition is normalized independently of the source-bound scientific-complex digest. Its stable identity contains the stable master-complex key plus the exact auxiliary vertices, tetrahedra, facet pairing, face coverage, and volumes. The broad-phase candidate-set digest is excluded because it is an implementation/evidence hash that can change when source-bound records are rebuilt without changing the exact partition.

Stage-10B search

Identity:

(master-complex key, master-partition key, compatibility-system key)

State records search completeness, counters, unresolved reasons, and the stable keys of every maximal generated tiling.

Natural tiling

Identity:

(stable scientific cell-complex identity, sorted selected PrimitiveRingKey set)

State records eligibility and all certification axes. The current `primitive_ring_bound` field is intentionally removed from state: changing K is the comparison coordinate, not a scientific change by itself.

Essential ring

Identity is the `PrimitiveRingKey`; state is the fact that the ring occurs as a face in at least one eligible natural tiling.

Snapshot completeness

`build_primitive_bound_snapshot()` produces a persistent `PrimitiveBoundSnapshot` with all stable records, source stage digests, the natural-tiling outcome, and a snapshot status.

A snapshot is `UNRESOLVED` when any of the following holds:

- primitive-ring enumeration is truncated or incomplete through K ;
- induced ring symmetry is incomplete through K ;
- any strength result is source-incomplete or resource-truncated;
- any supplied face search lacks a certified admissible witness;
- no Stage-10B search is supplied;
- any Stage-10B search is unresolved;
- the aggregate catalog contains unresolved candidates; or
- the caller supplies an explicit unresolved reason.

The unresolved reasons remain machine-readable. A finite certified weak ring or a complete `NONE` natural-tiling outcome is not unresolved merely because the scientific answer is negative.

Transition comparison

For consecutive bounds $K_i < K_{i+1}$, stable records are joined by

(category, key_digest)

and classified as:

- `ADDED`: absent at K_i , present at K_{i+1} ;
- `REMOVED`: present at K_i , absent at K_{i+1} ;
- `MODIFIED`: stable identity present at both bounds but with different state;
- `unchanged`: identical stable key and state digest.

The natural-tiling outcome kind (`NONE`, `UNIQUE`, or `MULTIPLE`) is compared separately.

Primitive-ring monotonicity

For one fixed topology, net view, embedding, search method, and complete lower-closed primitive search, increasing the upper bound cannot remove an already represented primitive ring key:

$$\mathcal{R}_{K_i} \subseteq \mathcal{R}_{K_{i+1}}.$$

If a ring stable key disappears between two COMPLETE snapshots, the transition is INVALID. This indicates a changed source definition, an implementation bug, or corrupted persistence. The report does not reinterpret the disappearance as a normal refinement change.

Transition status

INVALID	monotonicity or source-definition violation
UNRESOLVED	either endpoint is scientifically unresolved
CHANGED	complete endpoints differ scientifically
STABLE	complete endpoints have no stable-record or outcome changes

UNRESOLVED takes precedence over apparent equality but not over an explicit monotonicity violation.

Tested-suffix stabilization

For a tested sequence

$$K_0 < K_1 < \dots < K_n,$$

Stage 10C reports the earliest K_j in the final consecutive suffix for which all tested transitions

$$K_j \rightarrow K_{j+1} \rightarrow \dots \rightarrow K_n$$

are STABLE.

This is named `stable_tested_suffix_start`. It is intentionally not called a convergence proof. Stability at $K = 8, 10, 12$ means only that the supplied tested sequence did not change over that suffix; a larger untested primitive ring may still alter the result.

A single snapshot has no tested transition and therefore no stable suffix.

Public API

Resources

```
NaturalTilingRefinementResources(  
    max_bounds=16,  
    max_records_per_snapshot=1_000_000,  
    max_total_changes=2_000_000,  
)
```

Resource checks are transactional:

- `run_primitive_bound_refinement()` validates the number and order of bounds before invoking the rebuild callback;
- snapshot record count is checked before persistence;
- total transition changes are checked before the final report is returned.

No partial report is returned after a resource exception.

Build one snapshot

```
snapshot = build_primitive_bound_snapshot(build)
```

Input constraints:

- `build` must be a validated `PrimitiveBoundBuild`;
- all source digests must agree;
- stable records must remain within resource limits.

Output:

PrimitiveBoundSnapshot

Compare two bounds

```
transition = compare_primitive_bound_snapshots(lower, upper)
```

Input constraints:

- upper.primitive_ring_bound > lower.primitive_ring_bound;
- view, embedding, and topology digests are identical.

Output:

PrimitiveBoundTransition

Build a report from snapshots

```
report = build_primitive_bound_refinement_report(snapshots)
```

Snapshot bounds must be unique and strictly increasing.

Execute full rebuilds

```
report = run_primitive_bound_refinement(  
    bounds=(8, 10, 12),  
    rebuild=rebuild_for_bound,  
)
```

The callback contract is:

```
def rebuild_for_bound(K: int) -> PrimitiveBoundBuild:  
    ... # reconstruct every source-bound downstream stage
```

The callback is invoked exactly once per requested bound, in increasing order. It must return the same requested bound. Rebuild failures propagate; Stage 10C does not hide them as a stable result.

Algorithm

```
function refine(bounds, rebuild, resources):  
    validate bounds are nonempty, unique, strictly increasing  
    preflight len(bounds) <= max_bounds  
  
    snapshots = []  
    for K in bounds:  
        full = rebuild(K)  
        validate full.primitive_ring_bound == K  
        validate all downstream sources name full ring catalog  
        records = normalize_every_scientific_stage(full)  
        classify snapshot completeness  
        snapshots.append(canonical_snapshot(K, records))  
  
    transitions = []  
    for consecutive lower, upper:  
        join records by (category, stable key)  
        emit added, removed, and modified changes  
        if complete primitive ring disappeared:  
            mark INVALID  
        else if either endpoint unresolved:  
            mark UNRESOLVED  
        else if changes or outcome changed:  
            mark CHANGED  
        else:  
            mark STABLE
```

```
find final consecutive suffix of STABLE transitions
return canonical report
```

Let N_i be the stable-record count at bound K_i . Comparison uses hash-map joins and costs

$$O(N_i + N_{i+1})$$

expected time per transition, excluding canonical JSON construction and sorting. Persistence sorts records for deterministic serialization, giving $O(N_i \log N_i)$ worst-case ordering cost per snapshot.

The dominant scientific cost remains the caller's complete downstream rebuild, not Stage 10C comparison.

Serialization and tamper resistance

Every persistent object stores canonical JSON-derived SHA-256 digests:

- `StableRefinementRecord` validates identity and state digests;
- `PrimitiveBoundSnapshot` validates its complete payload digest;
- `PrimitiveBoundTransition` validates its change list and status digest; and
- `PrimitiveBoundRefinementReport.from_dict()` reconstructs snapshots and transitions, then recomputes every transition from the snapshots.

Serialized transitions are therefore not trusted as independent truth. Changing a status, change count, state digest, stable-suffix bound, or report digest is rejected.

Full source replay remains the responsibility of the per-stage source objects. Stage 10C persistence proves that the stored comparison is canonical for the stored snapshots; the `PrimitiveBoundBuild` validation proves source consistency when those snapshots are initially constructed.

Edge cases and failure semantics

Same scientific result, new catalog digest

Expected result: STABLE.

All source-bound stage digests may change while stable records remain identical. This is the central non-regression case.

New longer primitive rings with no tiling effect

Expected result: CHANGED because new ring records are added, even if the final natural-tiling outcome and essential rings remain unchanged. The report preserves both facts rather than collapsing refinement to a single outcome flag.

Strength changes after the component domain grows

Expected result: a MODIFIED strength record and any consequent face, search, or tiling changes. The target ring retains the same stable key.

Dense ring IDs are reordered

Expected result: no change, provided canonical ring keys and scientific states are unchanged.

Primitive ring disappears under complete refinement

Expected result: INVALID with a monotonicity witness.

Incomplete upper bound appears unchanged

Expected result: UNRESOLVED, not STABLE.

Different primitive cell, net view, or embedding

Expected result: input rejection. Cross-representation comparison requires an explicit mapping and is outside Stage 10C.

Multiple master refinements

All supplied master complexes, partitions, compatibility systems, and searches are represented independently. Stable keys deduplicate only exact scientific identities; incomparable alternatives remain present.

No Stage-10B search

The snapshot is retained as UNRESOLVED with an explicit reason. This permits inspection of upstream changes without falsely claiming a natural-tiling result.

Focused validation requirements

The Stage 10C gate must cover:

1. stable comparison across different primitive-ring catalog digests;
2. one independent callback invocation per increasing bound;
3. rejection of a bound/source mismatch;
4. primitive-ring disappearance as an invalid monotonicity violation;
5. modified strength state under one stable ring identity;
6. unresolved propagation despite apparent key stability;
7. transactional `max_bounds` preflight;
8. rejection of duplicate or decreasing bounds;
9. canonical report serialization round trip; and
10. tampered transition rejection.

The focused regression boundary also reruns Stages 4–10B so the refinement layer cannot weaken upstream source binding or natural-tiling semantics.

Deferred work

Stage 10C does not:

- generate master refinements from face candidates;
- map stable keys across changed topology graphs or primitive-cell gauges;
- prove asymptotic convergence beyond the tested bound sequence;
- choose a preferred candidate when MULTIPLE remains;
- label natural tiles geometrically or chemically; or
- certify the LTA tile set and 3 : 1 : 1 multiplicity ratio.

The next stage is Stage 10D, the LTA end-to-end validation gate at the declared primitive-bound sequence.

References

1. V. A. Blatov, O. Delgado-Friedrichs, M. O’Keeffe, and D. M. Proserpio, *Three-Periodic Nets and Tilings: Natural Tilings for Nets*, Acta Crystallographica Section A **63**, 418–425 (2007), DOI: 10.1107/S0108767307038287.
2. S. J. Chung, Th. Hahn, and W. E. Klee, *Nomenclature and Generation of Three-Periodic Nets: The Vector Method*, Acta Crystallographica Section A **40**, 42–50 (1984), DOI: 10.1107/S0108767384000088.